

ISLAMIC UNIVERSITY OF TECHNOLOGY

Assignment

Course: EEE 4506

Course Name: Numerical Methods Lab

Name: Mashrur Saad Emad
Student ID: 210021139

Name: Radiba Nuzhat
Student ID: 21002163

Section: A1

Date of Submission: 12-Mar-25

Introduction:

Nano-magnetism studies magnetic properties at the nanometer scale, where quantum mechanical effects and surface interactions dominate. A key framework in this field is the Landau-Lifshitz-Gilbert (LLG) equation, which describes magnetization dynamics in ferromagnetic materials.

Mathematical LLGS equation is expressed as:

$$\left(1+\alpha^2\right)\frac{d\hat{m}}{dt} = -\gamma\left(\hat{m}\times\hat{H}_{eff}\right) - \alpha\gamma\left(\hat{m}\times(\hat{m}\times\hat{H}_{eff}\right) \quad \text{.....(1)}$$

Here,

- $\hat{m}$ is the unit magnetization vector
- $\hat{H}_{eff}$ is the effective magnetic field containing all internal and external fields acting on the nanomagnet
- α is the Gilbert damping factor and the usually very small value for CoFeB is 0.01.
- γ is the gyromagnetic ratio for electrons and expressed as $\gamma = \frac{2\mu_b}{\hbar}$

For easier solving, we get the equation:

$$\frac{d\hat{m}}{d\tau} = -\left(\hat{m}\times\hat{h}\right) - \alpha(\hat{m}\times(\hat{m}\times\hat{h})$$

Where, $\tau = \frac{t\gamma H_k}{1+\alpha^2}$ and $\hat{h} = \frac{\hat{H}_{ext}}{H_k} + m_z\hat{z}$.

Here 3 tasks were given, based on this equation and parameters. The numerical technique used in the codes is the 4th order RK method.

The 4th order RK method formula is given by:

$y_{n+1} = y_n + \frac{1}{6}(k_1+2k_2+2k_3+k_4)$; where, $k_1 = f(x_i, y_i)$

$$k_2 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1h\right)$$

$$k_3 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2h\right)$$

$$k_4 = f(x_i + h, y_i + k_3h)$$

Task 1:

Here the given equation was solved using RK 4th order method instead of ode45 module.

```
clc;
clear all;
close all;

global alpha;
alpha = 0.01;
gamma = 2.21276e5;
mu = 1.2566e-6;
h_k = 3.1951e5;
m_initial=[0.01, 0.95, 0.01];
m_norm = m_initial/norm(m_initial);

tau1 = 0;
tau2 = 2000;
N = 1000;
h = (tau2 - tau1) / N;
tau = linspace(tau1, tau2, N+1);

m = zeros(N+1, 3);
m(1, :)=m_norm;

for i = 1:N
    k1 = h*llg(m(i, :), alpha);
    k2 = h*llg(m(i, :) + 0.5*k1, alpha);
    k3 = h*llg(m(i, :) + 0.5*k2, alpha);
    k4 = h*llg(m(i, :) + k3, alpha);

    m(i+1, :) = m(i, :) + (1/6) * (k1 + 2*k2 + 2*k3 + k4);
    m(i+1, :) = m(i+1, :) / norm(m(i+1, :));
end

time = tau * (1 + alpha^2) / (gamma * h_k);

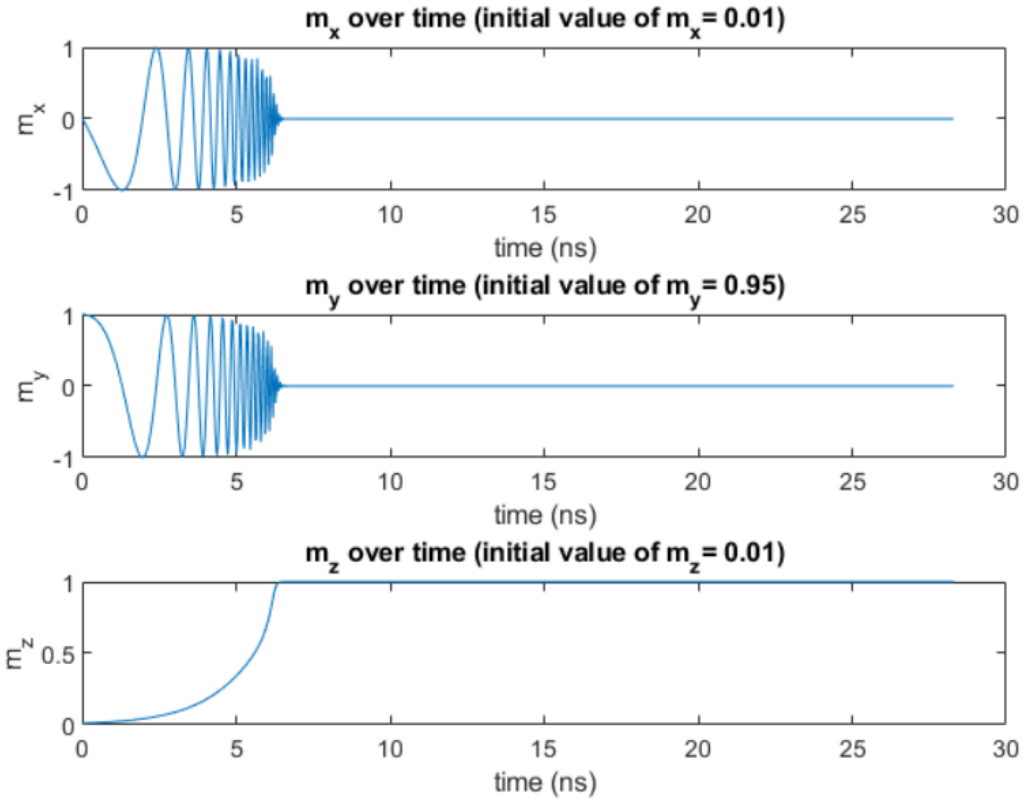
figure;
subplot(3,1,1);
plot(time * 1e9, m(:,1));
xlabel('time (ns)');
ylabel('m_x');
title(['m_x over time (initial value of m_x= ',num2str(m_initial(1)),')'])

subplot(3,1,2);
plot(time * 1e9, m(:,2));
xlabel('time (ns)');
ylabel('m_y');
title(['m_y over time (initial value of m_y= ',num2str(m_initial(2)),')'])
```

```
subplot(3,1,3);
plot(time * 1e9, m(:,3));
xlabel('time (ns)');
ylabel('m_z');
title(['m_z over time (initial value of m_z= ',num2str(m_initial(3)),')'])

function llg = llg(m, alpha)
H = [0, 0, m(3)];
dynamic = cross(m, H);
damping = cross(m, dynamic);
llg = -dynamic - alpha * damping;
end
```

Output:



Task 2:

```
clc;
clear;
close all;

global alpha;
alpha = 0.01;
gamma = 2.21276e5;
mu = 1.2566e-6;
h_k = 3.1951e5;

tau1 = 0;
tau2 = 2000;
N = 1000;
h = (tau2 - tau1) / N;
tau = linspace(tau1, tau2, N+1);

m_initial = input('initial magnetization position: ');

n = size(m_initial, 1);
relaxation_times = zeros(n, 1);

for j = 1:n
    m_norm = m_initial(j, :) / norm(m_initial(j, :));
    m = zeros(N+1, 3);
    m(1, :) = m_norm;

    for i = 1:N
        k1 = h*llg(m(i, :), alpha);
        k2 = h*llg(m(i, :) + 0.5*k1, alpha);
        k3 = h*llg(m(i, :) + 0.5*k2, alpha);
        k4 = h*llg(m(i, :) + k3, alpha);

        m(i+1, :) = m(i, :) + (k1 + 2*k2 + 2*k3 + k4) / 6;
        m(i+1, :) = m(i+1, :) / (norm(m(i+1, :)) );
    end

    time = tau * (1 + alpha^2) / (gamma * h_k);

    rt = find(abs(m(:, 3)) >= 0.98, 1);
    if isempty(rt)
        relaxation_times(j) = NaN;
    else
        relaxation_times(j) = time(rt) * 1e9;
    end

    figure;
    plot(time * 1e9, m(:,3))
```

```
    xlabel('Time (ns)');
    ylabel('m_z');
    title(['m_z over time for initial condition (m_x=', num2str(m_initial(j, 1)), ',
m_ay=', num2str(m_initial(j, 2)), ', m_z=', num2str(m_initial(j, 3)), ')']);

 end

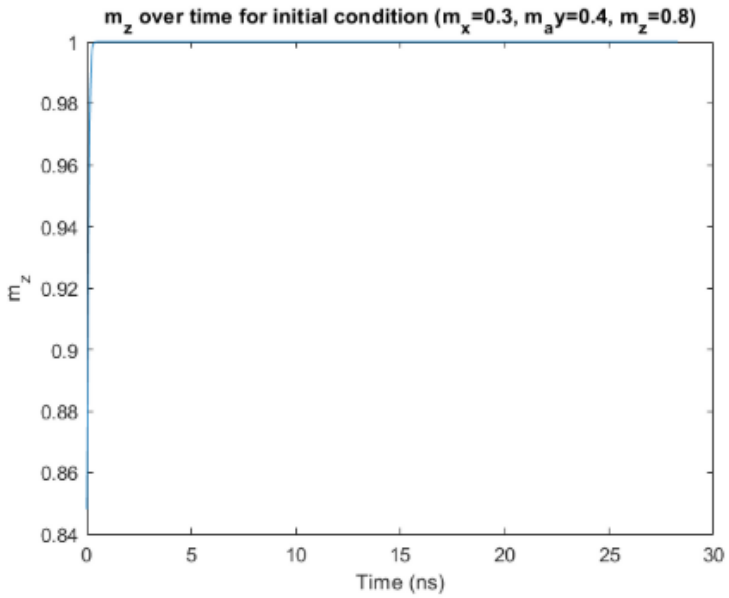
 disp(table(m_initial(:, 1), m_initial(:, 2), m_initial(:, 3),
relaxation_times,'VariableNames', {'m_x', 'm_y', 'm_z', 'Relaxation_Time_ns'}));

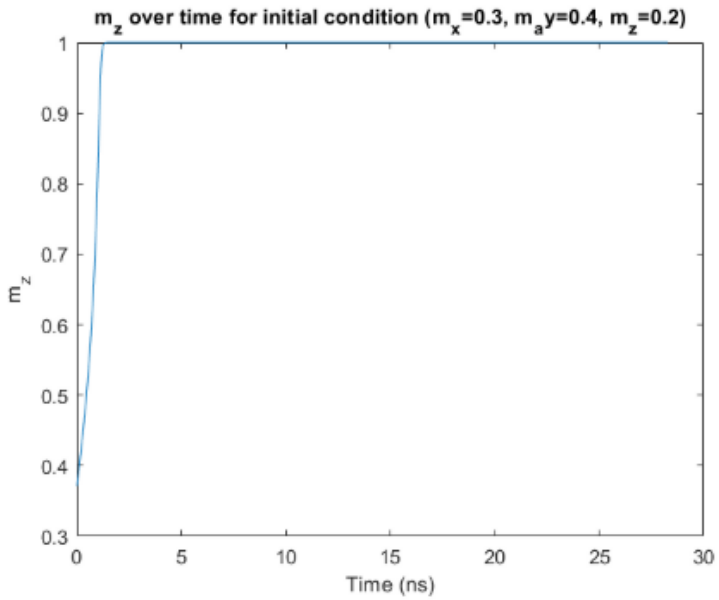
 function llg = llg(m, alpha)
 H = [0, 0, m(3)];
 dynamic = cross(m, H);
 damping = cross(m, dynamic);
 llg = -dynamic - alpha * damping;
 end
```

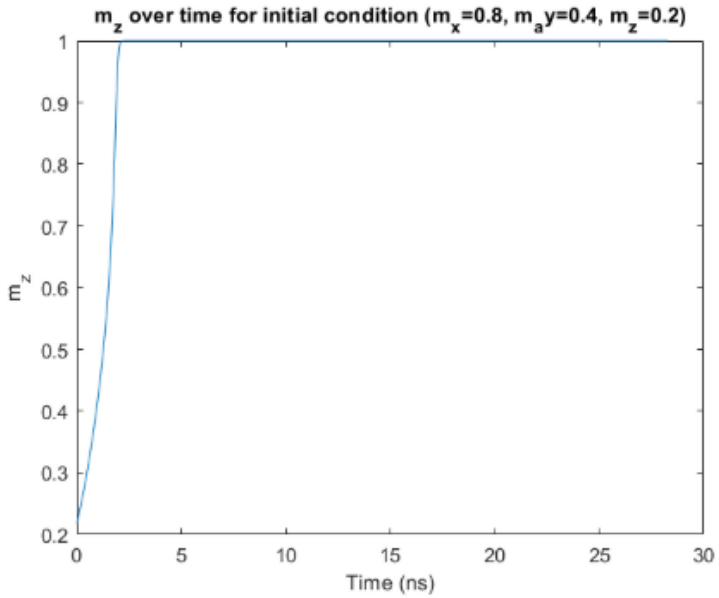
Command Window:

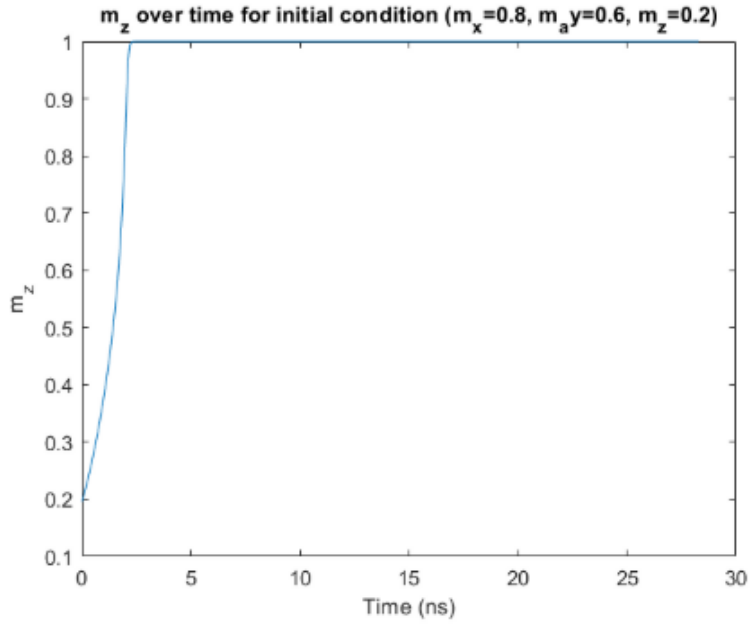
```
Command Window
  initial magnetization position: [0.3 0.4 0.8
  0.3 0.4 0.2
  0.8 0.4 0.2
  0.8 0.6 0.2
  0.3 0.4 -0.8
  0.3 0.4 -0.2]
```

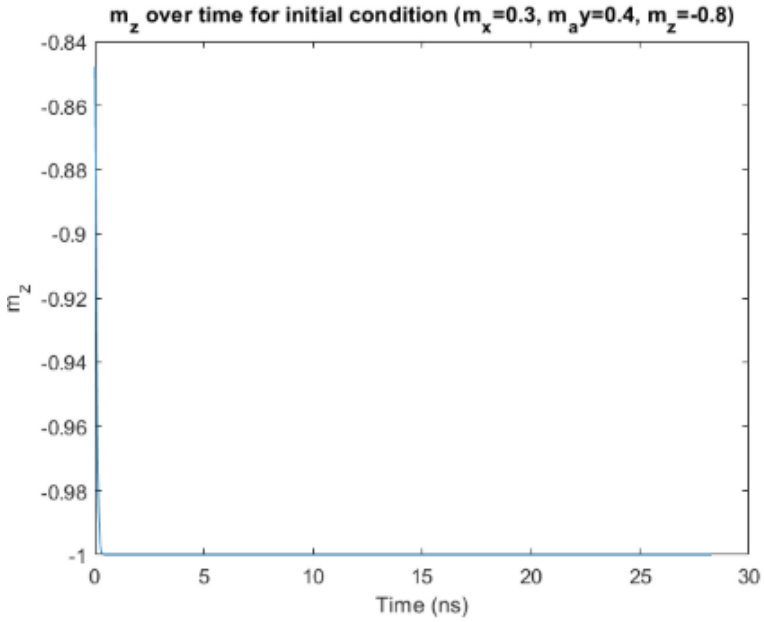
Output:

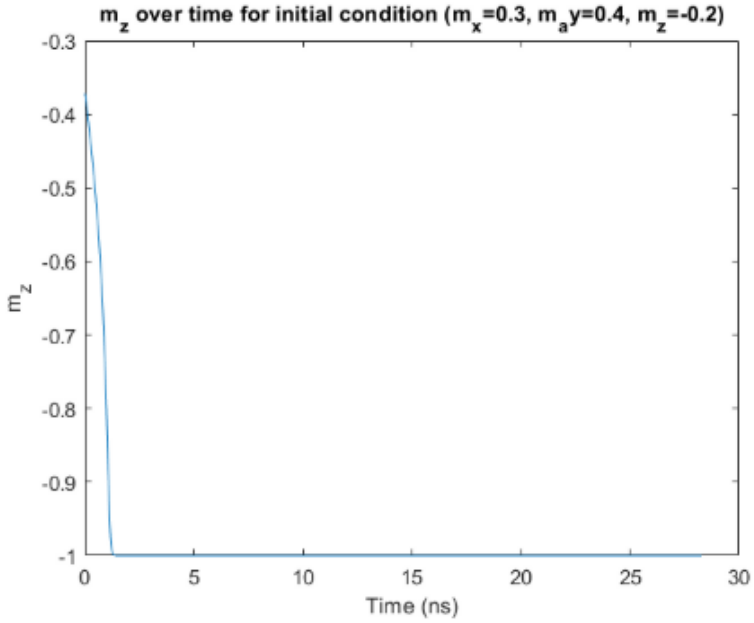

m_z over time for initial condition (m_x=0.3, m_ay=0.4, m_z=0.8)
m_z
Time (ns)


m_z over time for initial condition (m_x=0.3, m_ay=0.4, m_z=0.2)
m_z
Time (ns)


m_z over time for initial condition (m_x=0.8, m_ay=0.4, m_z=0.2)
m_z
Time (ns)


m_z over time for initial condition (m_x=0.8, m_ay=0.6, m_z=0.2)
m_z
Time (ns)


m_z over time for initial condition (m_x=0.3, m_ay=0.4, m_z=-0.8)
m_z
Time (ns)


m_z over time for initial condition (m_x=0.3, m_ay=0.4, m_z=-0.2)
m_z
Time (ns)

m_x	m_y	m_z	Relaxation_Time_ns
0.3	0.4	0.8	0.16975
0.3	0.4	0.2	1.1882
0.8	0.4	0.2	2.0087
0.8	0.6	0.2	2.1784
0.3	0.4	-0.8	0.16975
0.3	0.4	-0.2	1.1882

Comments:

We see from the table, comparing first two or the last two cases, keeping m_x & m_y constant, as m_z goes far from 1 or -1, the relaxation time increases.

Next, comparing the second three cases, we see keeping m_z constant, if m_x or m_z is far from 0, the relaxation time increases.

So, if m_z goes far from -1 or +1: relaxation time increases.

If m_x and m_y goes far from 0: relaxation time increases.

Task 3:

```
clc;
clc;
clear all;
close all;

gamma = 2.21276e5;
mu = 1.2566e-6;
h_k = 3.1951e5;

tau1 = 0;
tau2 = 2000;
N = 1000;
h = (tau2 - tau1) / N;
tau = linspace(tau1, tau2, N+1);

n=input("how many values of alpha (atleast 10): ");
alphas = linspace(0.001, 0.5, n);

relaxation_times = zeros(size(alphas));

m_initial = input("initial magnetization: ");
m_initial = m_initial/norm(m_initial);

for j = 1:length(alphas)
    alpha = alphas(j);

    m = zeros(N+1, 3);
    m(1, :) = m_initial;

    for i = 1:N
        k1 = h*llg(m(i, :), alpha);
        k2 = h*llg(m(i, :) + 0.5*k1, alpha);
        k3 = h*llg(m(i, :) + 0.5*k2, alpha);
        k4 = h*llg(m(i, :) + k3, alpha);

        m(i+1, :) = m(i, :) + (1/6) * (k1 + 2*k2 + 2*k3 + k4);
        m(i+1, :) = m(i+1, :) / norm(m(i+1, :));
    end

    time = tau * (1 + alpha.^2) / (gamma * h_k);

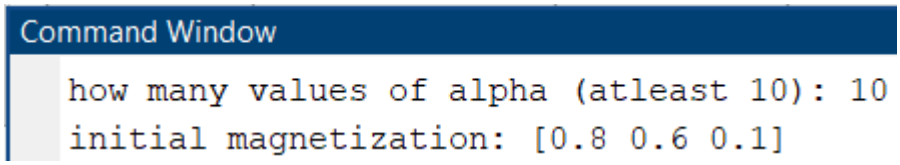
    figure;
    plot(time * 1e9, m(:,3));
    xlabel('Time (ns)');
    ylabel('m_z');
    title(['m_z over time for Different Gilbert Damping Factors (α = ' num2str(alpha,
'%.3f') ')']);
```

```
    rt = find(abs(m(:, 3)) >= 0.99, 1);
    if isempty(rt)
        relaxation_times(j) = NaN;
    else
        relaxation_times(j) = time(rt) * 1e9;
    end
end

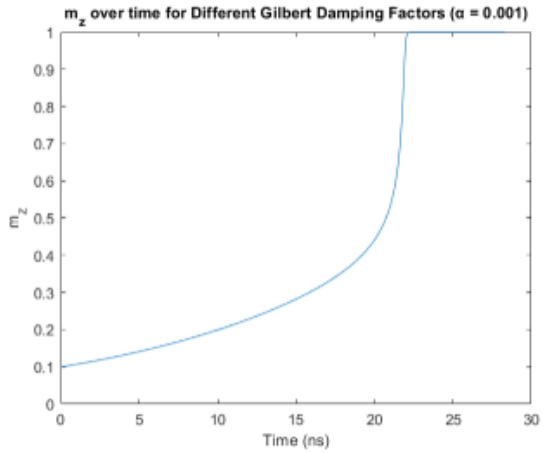
disp(table(alphas', relaxation_times','VariableNames', {'Damping Factor (α)',
'Relaxation Time (ns)'}));

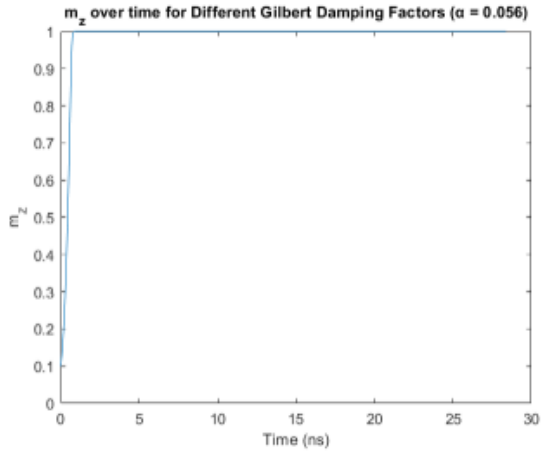
function llg = llg(m, alpha)
    H = [0, 0, m(3)];
    dynamic = cross(m, H);
    damping = cross(m, dynamic);
    llg = -dynamic - alpha * damping;
end
```

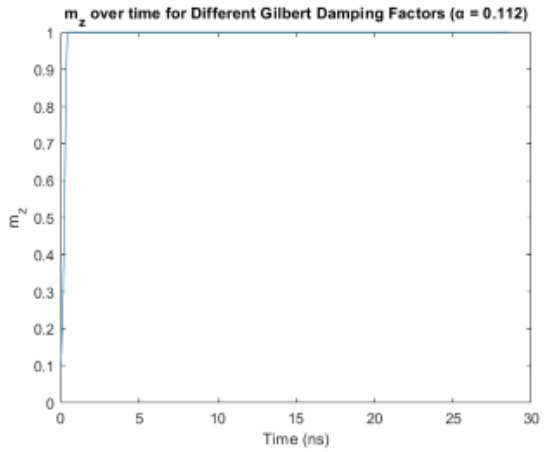
Command Window:

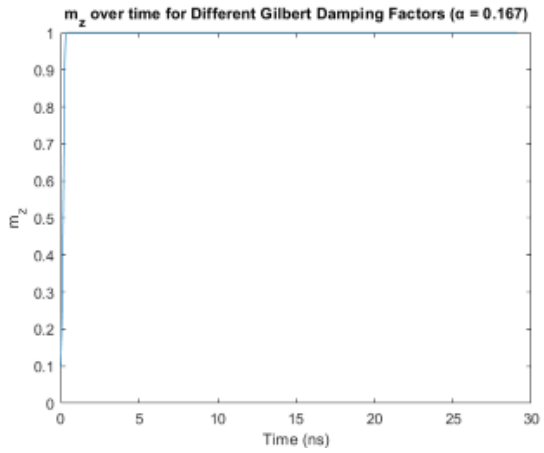


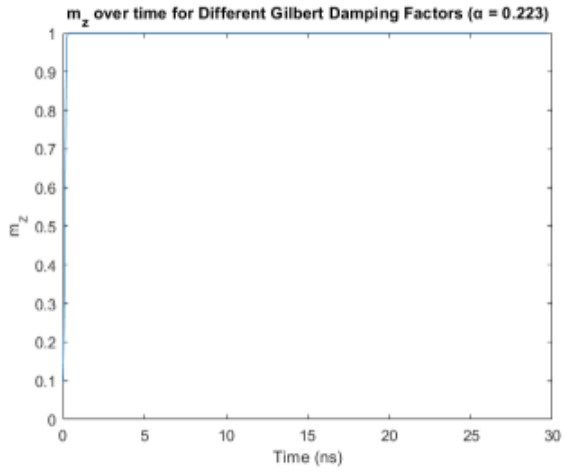
Output:

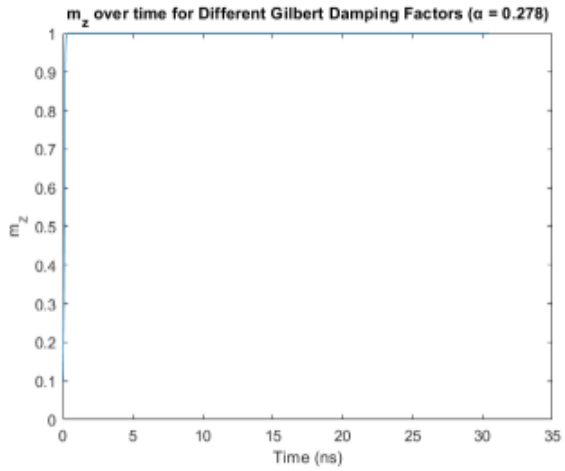

m_z over time for Different Gilbert Damping Factors (α = 0.001)
m_z
Time (ns)

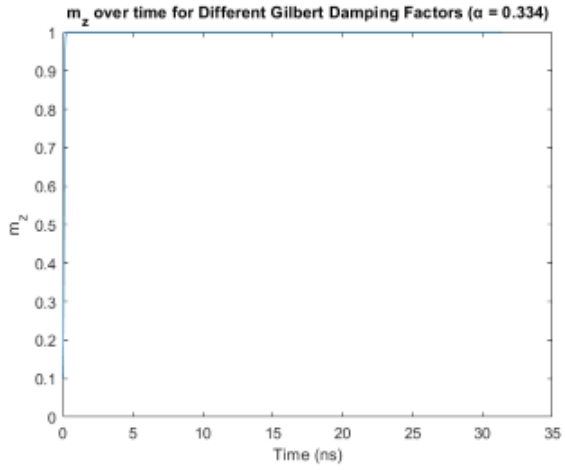

m_z over time for Different Gilbert Damping Factors (α = 0.056)
m_z
Time (ns)

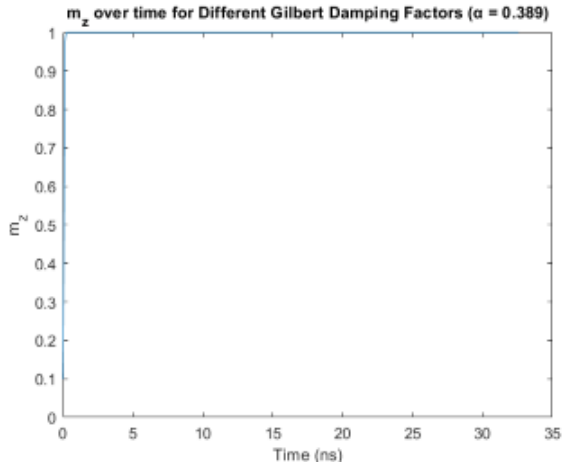

m_z over time for Different Gilbert Damping Factors (α = 0.112)
m_z
Time (ns)

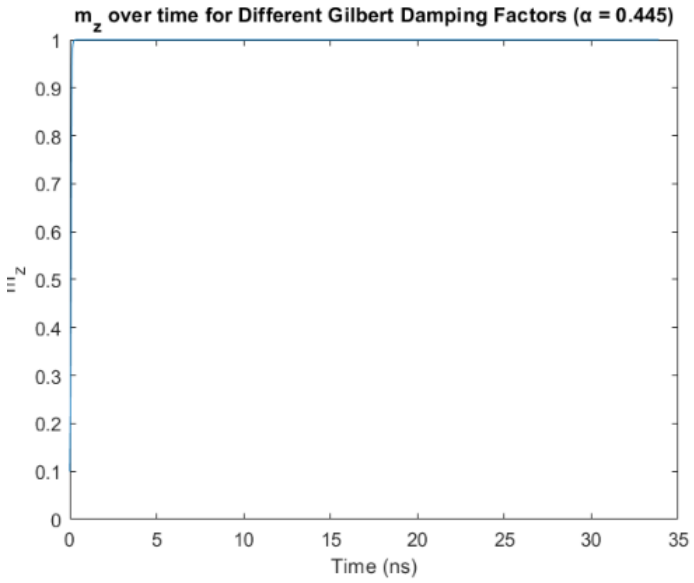

m_z over time for Different Gilbert Damping Factors (α = 0.167)
m_z
Time (ns)

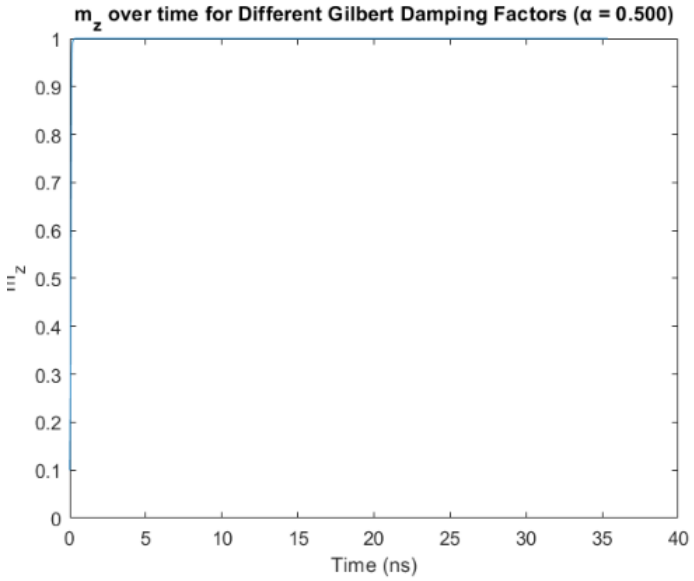

m_z over time for Different Gilbert Damping Factors (α = 0.223)
m_z
Time (ns)


m_z over time for Different Gilbert Damping Factors (α = 0.278)
m_z
Time (ns)


m_z over time for Different Gilbert Damping Factors (α = 0.334)
m_z
Time (ns)


m_z over time for Different Gilbert Damping Factors (α = 0.389)
m_z
Time (ns)





Damping Factor (α)	Relaxation Time (ns)
0.001	22.065
0.056444	0.76623
0.11189	0.45828
0.16733	0.31989
0.22278	0.26723
0.27822	0.24383
0.33367	0.22007
0.38911	0.19543
0.44456	0.20328
0.5	0.21216

Comments:

So we see, as the damping factor increases, the relaxation time decreases.

$\therefore Relaxation\ time = \frac{1}{\alpha}$

Discussion

All the tasks were done properly. The codes were shown and explained to the teacher. All the results, graphs and tables shown properly. There were comments added to wherever asked.

So it can be concluded that the assignment was done properly.